Table des matières

Exercices corrigés 10 — Formats de table ouverts, lakehouse et médaillon 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 4
Mises en situation 4
Corrigé 5

Exercices corrigés 10 — Formats de table ouverts, lakehouse et médaillon

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur les tables que vous venez d'écrire et sur les chiffres que vous avez sous les yeux ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 10 et du Labo 10.

Les chiffres à retrouver. Les questions pratiques supposent le labo fini, jusqu'à l'étape 9 : la télémétrie en Iceberg compactée, expirée et nettoyée, les trois séries de l'argent chargées deux fois, et la table Delta compactée et nettoyée. Les requêtes se tapent dans la ligne de commande DuckDB, lancée depuis `Projet_Boreal`, après `.read 01_de/sql/lac_iceberg.sql` ; les commandes Python, dans l'Anaconda Prompt, depuis `01_de`. La pratique 9 pose une étiquette : faites-la en dernier.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Fichier ou table. Dites en une phrase ce qu'un format de fichier range, puis ce qu'un format de table ajoute.
2. Quatre manques. Nommez les quatre choses qu'un dossier de fichiers Parquet ne sait pas faire, et donnez pour chacune ce qui se passe chez Boréal.
3. L'arbre. Nommez, du haut vers le bas, les cinq niveaux qu'une lecture d'une table Iceberg traverse, et dites ce que porte chacun.
4. Le journal. Nommez les actions qu'un fichier du journal de Delta peut porter, et dites laquelle paraît dans la version de mai 2025 de la télémétrie, et pourquoi.
5. Quatre lettres. Développez ACID, et dites, pour chaque lettre, ce qui la donne dans un format de table.

© 2026 iAlexMG. Tous droits réservés.

6. Deux écrivains. Deux programmes ajoutent en même temps à la même table Iceberg. Dites ce qui arrive au second au moment de valider, et ce que PyIceberg 0.12 en fait.
7. La colonne 9. Dites pourquoi le fichier de mars 2024 se relit sous le nom `temperature_moteur_max` alors qu'il porte `temp_moteur_max`, et ce qu'il rend pour la colonne 20.
8. Élargir ou rétrécir. Donnez deux changements de type qu'Iceberg accepte sans réécrire, et dites ce qu'il fait d'un changement qui rétrécit.
9. Caché ou par chemin. Dites ce qui distingue le partitionnement caché d'Iceberg de celui du Cours 02, du point de vue de celui qui écrit la requête.
10. Trois références. Définissez un instantané, une branche et une étiquette, et dites ce que l'expiration fait de chacun par défaut.
11. Deux horloges. Nommez les deux horloges d'une table, et dites laquelle le voyage dans le temps suit.
12. Trois gestes. Dans quel ordre compacte-t-on, expire-t-on et nettoie-t-on une table Iceberg, et que libère chacun ?
13. Trois métaux. Définissez le bronze, l'argent et l'or, et dites qui peut reconstruire chaque couche.

Questions pratiques

1. [labo] Mois par mois. Écrivez la requête qui compte les messages de la table Iceberg par mois de

   2025. Nommez le mois le plus chargé, et dites où, dans le labo, vous auriez pu trouver les mêmes comptes sans rien interroger.
2. [labo] Un fichier par mois. Par `iceberg_metadata`, comptez les fichiers de données et les lignes de chaque partition de la table courante, et montrez les cinq mois les plus récents. Dites combien de fichiers porte chaque partition, et pourquoi.

1. [labo] Les heures du moteur. Comptez, par micrologiciel, les messages de la table Iceberg et ceux qui portent les heures du moteur. Expliquez le résultat par l'évolution du schéma.
2. [labo] Le journal compté. Lisez tous les fichiers du journal de la table Delta d'une seule requête, et comptez les fichiers ajoutés, les fichiers retirés, et les lignes ajoutées d'après les statistiques. Expliquez pourquoi les lignes ajoutées sont le double des messages.
3. [labo] La production. Rangez dans une variable le fichier de métadonnées courant de `argent.production_horaire`, puis prouvez que la table Iceberg égale celle de l'entrepôt, dans les deux sens.
4. [labo] L'énergie en UTC. Pour `argent.energie_horaire`, donnez le nombre de partitions, la première et la dernière, puis le nombre de lignes de la dernière. Expliquez ce qu'elle contient.
5. [labo] Ce que l'overwrite laisse. Pesez, par sorte de fichier, le dossier de `argent.usine_horaire`. Dites pourquoi il porte deux fois plus de fichiers de données que la table n'en cite.

© 2026 iAlexMG. Tous droits réservés.

6. [labo] Une vue réconciliée. Dans la ligne de commande DuckDB, créez une vue sur la table Delta qui ne montre qu'une colonne de température, remplie pour tous les messages, et vérifiez-la.

7. [labo] Une deuxième étiquette. Étiquetez l'état courant `fin-mars-2026`, lisez les références de la table, puis tentez d'expirer. Relisez l'étiquette, et dites ce que l'expiration a épargné, et pourquoi.

Diagnostics

1. Un analyste copie le dossier `plateforme/lac/iceberg/bronze/telemetrie` sur un autre disque pour travailler hors ligne, et la lit par son fichier de métadonnées : 204 485 messages, tout va bien. De retour au bureau, il efface l'original pour faire de la place, et sa copie ne se lit plus : `IO Error: Cannot open file ".../telemetrie/metadata/snap-….avro"`. Expliquez les deux résultats.

2. Pour aller vite, un développeur lit les fichiers de la télémétrie directement, sans passer par la table :

   ```
   SELECT COUNT(*) AS lignes, COUNT(temp_moteur_max) AS temp_moteur_max,
          COUNT(temperature_moteur_max) AS temperature_moteur_max
   FROM read_parquet('plateforme/lac/iceberg/bronze/telemetrie/data/**/*.parquet',
                     union_by_name = true);
   ```

   Il obtient 204 485 lignes, mais deux colonnes de température, l'une remplie 78 843 fois et l'autre 125 642 fois. Le lendemain du compactage, avant le nettoyage, la même requête rend 250 824 lignes. Expliquez les deux résultats.

3. Une gestionnaire veut les chiffres publiés au 31 décembre 2025. Elle lit la table par `snapshot_from_timestamp => TIMESTAMP '2025-12-31 23:59:59'`, obtient 0 message sans aucune erreur, et conclut que la télémétrie de 2025 a été perdue. Expliquez, et dites ce qu'elle aurait dû lire.

4. Un ingénieur fait le ménage : il lance `expirer --garder 1` sur la table de la télémétrie, avant que personne ait posé l'étiquette `fin-2025`. Dites ce qui est perdu, ce qui ne l'est pas, et ce qu'il aurait fallu faire.

5. Pendant qu'un chargement quotidien de la télémétrie tourne, une collègue lance `nettoyer --age 0` pour libérer de l'espace. Le chargement échoue au moment de valider. Expliquez pourquoi, et pourquoi le nettoyage a une valeur par défaut de 72 heures.

6. Un auditeur demande l'état de la table Delta au 31 décembre 2025, la version 27. La requête rend `IO Error: Cannot open file "…/part-00000-….snappy.parquet": Le fichier spécifié est introuvable.` Expliquez ce qui s'est passé, et donnez deux façons de l'éviter.

7. Une développeuse réécrit `argent_iceberg.py` et retire la ligne `SET TimeZone = 'UTC'`, qu'elle juge inutile. Le programme s'arrête sur `UnsupportedPyArrowTypeException : Column 'heure' has an unsupported type: timestamp[us, tz=America/Toronto]`. Expliquez, et dites pourquoi remettre la ligne ne change aucun instant.

8. Pour estimer le volume de mars 2026 dans `argent.usine_horaire`, un planificateur additionne les lignes des fichiers de la partition `mois=2026-03` : 40 920, soit 744 heures pleines pour les 55 étiquettes. Une requête sur mars, à l'heure de l'Est, en rend 40 865. Expliquez l'écart de 55 lignes.

© 2026 iAlexMG. Tous droits réservés.

Questions à développer

A. Boréal doit choisir un format pour son lac : Iceberg ou Delta Lake. En vous appuyant sur ce que le labo a mesuré, et sur ce que les documentations disent de Snowflake, de Databricks et de Fabric, défendez un choix en une page. Dites ce qui le ferait changer.

B. « Avec le voyage dans le temps, on n'a plus besoin de sauvegardes. » Montrez, à partir des étapes 7 et 9 du labo, pourquoi c'est faux, puis dites ce que le voyage dans le temps change au droit à l'effacement d'un client.

C. Le parcours donne à Data Engineering l'argent des séries de l'usine, mais laisse les sources relationnelles passer de la `RAW` au `STAGING` d'Analytics Engineering. Justifiez ce partage, et dites ce qui arriverait si Data Engineering tenait aussi un argent des commandes de l'ERP.

Mises en situation

1. Une coopérative de pêcheurs fictive de Rimouski reçoit les journaux de bord électroniques de ses bateaux, un fichier par sortie, dans un dossier de Parquet partitionné par jour. Au printemps, le fournisseur du journal de bord publie une version qui renomme `poids_kg` en `masse_debarquee_kg` et ajoute la température de l'eau. La moitié de la flotte passera à la nouvelle version en deux mois. Proposez une façon de ranger ces données pour que les lecteurs ne voient qu'une colonne, et dites ce que vous demanderez à l'outil qui écrit.

2. Le port fictif de Sept-Îles garde en table Iceberg les passages de ses navires, et un règlement l'oblige à reproduire, pendant cinq ans, les chiffres de chaque rapport trimestriel tels qu'ils ont été publiés. La table reçoit des corrections chaque semaine. Un armateur demande en même temps l'effacement des renseignements personnels de ses équipages. Proposez une politique d'étiquettes, d'expiration et de nettoyage, et dites comment elle concilie les deux demandes.

3. Une aluminerie fictive de Baie-Comeau monte un lakehouse. Ses cuves remontent leurs mesures par un historien ; son ERP livre des extraits mensuels ; ses analystes veulent des tableaux de bord de production. Rangez chaque source en bronze, argent et or, et dites qui, de l'ingénieur de données ou de l'ingénieur analytique, tient chaque couche.

4. (avancé) Une chaîne de quincailleries fictive de Shawinigan tient ses ventes dans un dossier de Parquet qu'un seul analyste lit par DuckDB. Un consultant lui propose une plateforme clés en main, Fabric ou Databricks ; un autre, des tables Iceberg locales avec un catalogue SQL. Comparez les trois voies selon ce qu'elles coûtent en entretien, ce qu'elles ouvrent à d'autres moteurs, et ce dont la chaîne a vraiment besoin aujourd'hui, puis recommandez-en une.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

Les chiffres cités sont ceux de la génération du cours, jusqu'à 2026-03, et du labo rejoué dans l'ordre du lecteur. Les identifiants d'instantanés, les noms des fichiers de métadonnées et les durées changent d'un passage à l'autre ; les comptes, eux, ne changent pas.

Questions courtes

1. Fichier ou table. Un format de fichier, Parquet par exemple, range des lignes dans un fichier : colonnes, compression, statistiques. Un format de table dit quels fichiers forment la table à un instant donné, sous quel schéma et quelle partition, et publie chaque nouvel état d'un seul geste.

Théorie : `Theorie_10_DE.md`, section 1.

2. Quatre manques. Valider d'un coup : un lecteur voit une écriture à moitié faite, deux écrivains s'écrasent ; chez Boréal, le bronze se réécrit par jour livré. Changer de schéma : renommer veut dire tout réécrire ou garder deux noms ; le bronze porte `temp_moteur_max` et `temperature_moteur_max` côte à côte. Se souvenir : un fichier réécrit remplace l'ancien sans trace. Se décrire : le schéma et les statistiques se découvrent en ouvrant les 933 fichiers.

Théorie : `Theorie_10_DE.md`, section 1.

3. L'arbre. Le catalogue, qui tient le chemin du fichier de métadonnées courant ; le fichier de métadonnées, qui porte les schémas, la partition, les instantanés et les étiquettes ; la liste de manifestes d'un instantané, qui nomme ses manifestes avec la plage de leurs partitions ; le manifeste, qui liste des fichiers de données avec leurs statistiques ; le fichier de données, en Parquet.

Théorie : `Theorie_10_DE.md`, section 2.

4. Le journal. `add`, un fichier qui entre, avec ses statistiques ; `remove`, un fichier qui sort ; `metaData`, le schéma et les propriétés quand ils changent ; `commitInfo`, qui a validé quoi et quand ; plus le protocole, dans les premières versions. La version 20, celle de mai 2025, porte `commitInfo`, `metaData` et `add` : elle ajoute un fichier, et la fusion de schéma y a ajouté les colonnes `heures_moteur` et `temperature_moteur_max`.

Théorie : `Theorie_10_DE.md`, section 2.

5. Quatre lettres. Atomicité : un seul pointeur change, et les fichiers écrits avant restent invisibles jusque-là. Cohérence : le schéma est dans les métadonnées, et l'écrivain s'y conforme. Isolation : un lecteur suit un fichier de métadonnées fixe et ne voit rien de ce qui se valide ensuite. Durabilité : tout est écrit en fichiers avant que le pointeur bouge.

Théorie : `Theorie_10_DE.md`, section 3.

6. Deux écrivains. Le second demande au catalogue de remplacer le pointeur à condition qu'il n'ait pas bougé ; le premier l'a déjà remplacé, et la validation est refusée. PyIceberg 0.12 écrit `Commit failed due to a concurrent update, retrying (1/4) in 102 ms`, refait son ajout sur l'état neuf, et valide : les deux ajouts sont dans la table. C'est la concurrence optimiste.

Théorie : `Theorie_10_DE.md`, section 3.

© 2026 iAlexMG. Tous droits réservés.

7. La colonne 9. Le lecteur ne cherche pas un nom, il cherche l'identifiant 9, que le fichier porte sous l'ancien nom et que le schéma 1 appelle `temperature_moteur_max`. La colonne 20, `heures_moteur`, est née avec le schéma 1 : le fichier de 2024 ne la contient pas, et le lecteur rend `NULL`.

Théorie : `Theorie_10_DE.md`, section 4.

8. Élargir ou rétrécir. `int` vers `long`, `float` vers `double`, ou un décimal vers une précision plus grande, sans réécriture : les anciens fichiers se convertissent au vol. Un changement qui rétrécit est refusé, puisque des valeurs déjà écrites pourraient ne plus tenir.

Théorie : `Theorie_10_DE.md`, section 4.

9. Caché ou par chemin. Avec le partitionnement par chemin du Cours 02, la partition est une colonne que l'écrivain calcule et sur laquelle le lecteur doit filtrer pour sauter des fichiers. Avec le partitionnement caché, la table dérive la partition d'une colonne ordinaire, `jour`, par une transformation ; le lecteur filtre sur `jour` comme d'habitude, et la table saute les fichiers d'elle-même.

Théorie : `Theorie_10_DE.md`, section 5.

10. Trois références. Un instantané est l'état complet de la table après une validation. Une branche est une lignée d'instantanés qui avance à chaque validation, `main` pour l'état courant. Une étiquette est un nom fixe posé sur un instantané. Par défaut, l'expiration oublie les vieux instantanés, mais jamais ceux qu'une branche ou une étiquette désigne, et ni les branches ni les étiquettes n'expirent.

Théorie : `Theorie_10_DE.md`, sections 6 et 7.

11. Deux horloges. Celle des faits, dans les colonnes de la table (`jour`, `recu_le`), et celle de la vie de la table, l'heure de chaque validation. Le voyage dans le temps ne suit que la seconde.

Théorie : `Theorie_10_DE.md`, section 6.

12. Trois gestes. Compacter d'abord, qui réécrit les petits fichiers en gros et ne libère rien : les anciens restent cités par les instantanés d'avant. Expirer ensuite, qui oublie ces instantanés et libère les métadonnées de leurs références ; chez Spark, il efface aussi les fichiers qu'ils étaient seuls à citer, pas chez PyIceberg 0.12. Nettoyer enfin, qui efface les fichiers que plus rien ne cite. Au labo : 90 fichiers en 3, 122 instantanés expirés, 90 fichiers effacés.

Théorie : `Theorie_10_DE.md`, section 7.

13. Trois métaux. Le bronze garde la donnée telle qu'elle est arrivée ; personne ne le reconstruit, c'est la copie de la source. L'argent la garde nettoyée et conforme ; il se reconstruit à partir du bronze. L'or la prépare pour un usage ; il se reconstruit à partir de l'argent.

Théorie : `Theorie_10_DE.md`, section 9.

Questions pratiques

1. [labo] Mois par mois.

© 2026 iAlexMG. Tous droits réservés.

```
SELECT DATE_TRUNC('month', jour) AS mois, COUNT(*) AS messages
FROM iceberg_scan(GETVARIABLE('telemetrie'))
WHERE jour BETWEEN DATE '2025-01-01' AND DATE '2025-12-31'
GROUP BY ALL ORDER BY ALL;
```

Douze lignes, de 5 727 messages en janvier à 13 344 en août, le mois le plus chargé ; juillet suit avec 12 673. Les mêmes comptes sont dans l'historique de la table Delta, à l'étape 9 : la dernière colonne y donne les lignes ajoutées par chaque version, un mois par version. Dans l'historique d'Iceberg, il faudrait soustraire deux totaux successifs.

Théorie : `Theorie_10_DE.md`, section 6.

2. [labo] Un fichier par mois.

```
SELECT REGEXP_EXTRACT(file_path, 'mois=([0-9-]+)', 1) AS mois, COUNT(*) AS fichiers,
       SUM(record_count) AS lignes
FROM iceberg_metadata(GETVARIABLE('telemetrie'))
WHERE status <> 'DELETED'
GROUP BY ALL ORDER BY mois DESC LIMIT 5;
```

mois varchar	fichiers int64	lignes int128
2026-03	1	13114
2026-02	1	17149
2026-01	1	16076
2025-12	1	9386
2025-11	1	7386

Chaque partition porte un fichier : les mois de 2023 à 2025 ont été chargés d'un ajout chacun, et les trois mois de 2026, chargés jour par jour, ont été compactés à l'étape 7. Le filtre sur le statut écarte les entrées d'un manifeste qui enregistrent un retrait.

Théorie : `Theorie_10_DE.md`, sections 2 et 7.

3. [labo] Les heures du moteur.

```
SELECT micrologiciel, COUNT(*) AS messages, COUNT(heures_moteur) AS heures
FROM iceberg_scan(GETVARIABLE('telemetrie'))
GROUP BY ALL ORDER BY ALL;
```

micrologiciel varchar	messages int64	heures int64

© 2026 iAlexMG. Tous droits réservés.

```
┌──────────────────┬──────────────┬──────────┐
│                  │              │          │
├──────────────────┼──────────────┼──────────┤
│ 1.3.0            │        71029 │        0 │
│ 1.6.2            │        17649 │        0 │
│ 2.0.1            │       115807 │   115807 │
└──────────────────┴──────────────┴──────────┘
```

La colonne 20 n'existe que depuis le schéma 1. Les fichiers écrits avant n'en ont pas, et les messages des véhicules restés en 1.x après le 1er mai 2025 n'en portaient pas : tous rendent `NULL`. Seuls les messages du micrologiciel 2.0 ont les heures du moteur, tous.

Théorie : `Theorie_10_DE.md`, section 4.

4. [labo] Le journal compté.

```sql
SELECT COUNT(add) AS ajouts, COUNT(remove) AS retraits,
       SUM(CAST(JSON_EXTRACT(add.stats, '$.numRecords') AS BIGINT)) AS lignes_ajoutees
FROM read_json('plateforme/lac/delta/telemetrie/_delta_log/*.json', union_by_name = true);
```

```
┌──────────┬──────────┬──────────────────┐
│ ajouts   │ retraits │ lignes_ajoutees  │
│ int64    │  int64   │      int128      │
├──────────┼──────────┼──────────────────┤
│       32 │       31 │           408970 │
└──────────┴──────────┴──────────────────┘
```

Trente et un fichiers ajoutés par les écritures mensuelles, puis un par le compactage, qui a retiré les 31 autres. Le compactage réécrit toutes les lignes dans son fichier neuf : 204 485 à l'écriture, 204 485 au compactage, 408 970 en tout. Le journal additionne des actions, pas des lignes vivantes ; l'état de la table est ce qui reste une fois les retraits soustraits.

Théorie : `Theorie_10_DE.md`, sections 2 et 7.

5. [labo] La production.

```sql
SET VARIABLE production = (SELECT metadata_location FROM catalogue.iceberg_tables
                           WHERE table_namespace = 'argent' AND table_name = 'production_horaire');
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
SELECT (SELECT COUNT(*) FROM (FROM iceberg_scan(GETVARIABLE('production'))
                              EXCEPT ALL FROM entrepot.argent.production_horaire)) AS en_trop,
       (SELECT COUNT(*) FROM (FROM entrepot.argent.production_horaire
                              EXCEPT ALL FROM iceberg_scan(GETVARIABLE('production')))) AS manquants;
```

0 en trop, 0 manquant, sur 13 104 lignes. `EXCEPT ALL` garde la multiplicité : une ligne en double d'un seul côté se verrait.

Théorie : `Theorie_10_DE.md`, section 9.

© 2026 iAlexMG. Tous droits réservés.

6. [labo] L'énergie en UTC.

```
SET VARIABLE energie = (SELECT metadata_location FROM catalogue.iceberg_tables
                        WHERE table_namespace = 'argent' AND table_name = 'energie_horaire');
SELECT COUNT(DISTINCT REGEXP_EXTRACT(file_path, 'mois=([0-9-]+)', 1)) AS partitions,
       MIN(REGEXP_EXTRACT(file_path, 'mois=([0-9-]+)', 1)) AS premiere,
       MAX(REGEXP_EXTRACT(file_path, 'mois=([0-9-]+)', 1)) AS derniere
FROM iceberg_metadata(GETVARIABLE('energie'))
WHERE status <> 'DELETED';
SELECT COUNT(*) AS lignes, MIN(heure), MAX(heure)
FROM iceberg_scan(GETVARIABLE('energie'))
WHERE heure >= TIMESTAMPTZ '2026-04-01 00:00:00+00';
```

52 partitions, de `2022-01` à `2026-04`, pour une série qui va du 1er janvier 2022 au 31 mars 2026 : 51 mois, plus une partition d'avril. Elle porte 4 lignes, les heures de 20 h à 23 h le 31 mars, heure de l'Est, qui sont déjà le 1er avril en temps universel. La première heure, le 1er janvier 2022 à minuit à l'heure de l'Est, est 5 h à Greenwich : elle reste en janvier.

Théorie : `Theorie_10_DE.md`, section 5.

7. [labo] Ce que l'overwrite laisse.

```
SELECT CASE WHEN filename LIKE '%.parquet' THEN 'données'
            WHEN filename LIKE '%.metadata.json' THEN 'métadonnées'
            WHEN PARSE_FILENAME(filename) LIKE 'snap-%' THEN 'listes de manifestes'
            ELSE 'manifestes' END AS sorte,
       COUNT(*) AS fichiers, ROUND(SUM(size) / 1e6, 2) AS mo
FROM read_blob('plateforme/lac/iceberg/argent/usine_horaire/**/*')
GROUP BY ALL ORDER BY ALL;
```

sorte varchar	fichiers int64	mo double
données	14	1.72
listes de manifestes	3	0.01
manifestes	3	0.02
métadonnées	4	0.01

Quatorze fichiers de données pour les 7 que la table cite. Le second passage a remplacé la table par un `overwrite`, qui fait deux instantanés, l'un qui retire et l'autre qui ajoute ; les 7 fichiers du premier passage restent sur le disque, cités par le premier instantané, et le resteront jusqu'à l'expiration et au nettoyage.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_10_DE.md`, section 7.

8. [labo] Une vue réconciliée.

```
LOAD delta;
CREATE OR REPLACE VIEW telemetrie_delta AS
SELECT * EXCLUDE (temp_moteur_max, temperature_moteur_max),
       COALESCE(temperature_moteur_max, temp_moteur_max) AS temperature_moteur_max
FROM delta_scan('plateforme/lac/delta/telemetrie');
SELECT COUNT(*) AS messages, COUNT(temperature_moteur_max) AS temperature
FROM telemetrie_delta;
```

204 485 messages, et 204 485 températures : chaque message a sa valeur, sous l'un ou l'autre nom. La vue ne vit que dans la session DuckDB : elle ne change rien à la table, et chaque moteur qui lit la table Delta doit refaire le même `COALESCE`. C'est ce que le renommage d'Iceberg évite.

Théorie : `Theorie_10_DE.md`, section 4.

9. [labo] Une deuxième étiquette. Depuis `01_de` :

```
python -m tables.entretien_iceberg etiqueter fin-mars-2026 --rang 2
python -m tables.entretien_iceberg expirer --garder 1
```

```
L'instantané de rang 2 (8947001000568374103) s'appelle maintenant fin-mars-2026.
La table : 2 instantanés, 31 fichiers de données cités, 31 sur le disque.
0 instantanés expirés ; --garder 1 et les étiquettes épargnent le reste.
La table : 2 instantanés, 31 fichiers de données cités, 31 sur le disque.
```

Puis, dans la ligne de commande DuckDB, après `.read 01_de/sql/lac_iceberg.sql` :

```
SELECT refs FROM read_json(GETVARIABLE('telemetrie'));
SET VARIABLE fin_mars = (SELECT refs['fin-mars-2026']['snapshot-id']
                         FROM read_json(GETVARIABLE('telemetrie')));
SELECT COUNT(*) AS messages, MAX(jour) AS dernier_jour
FROM iceberg_scan(GETVARIABLE('telemetrie'), snapshot_from_id => GETVARIABLE('fin_mars'));
```

Trois références : `main`, `fin-2025` et `fin-mars-2026`, dont deux désignent le même instantané, l'état courant. L'étiquette rend 204 485 messages, jusqu'au 2026-03-31. L'expiration n'a rien oublié : les deux instantanés qui restent sont étiquetés, et le dernier l'est deux fois. Une étiquette ne coûte rien à poser ; elle coûte ce qu'elle garde, tant qu'elle existe.

Théorie : `Theorie_10_DE.md`, sections 6 et 7.

Diagnostics

© 2026 iAlexMG. Tous droits réservés.

1. Les métadonnées d'Iceberg citent leurs fichiers par leur chemin absolu. La copie se lisait, mais par les fichiers de l'original : `iceberg_metadata` sur la copie donne des chemins qui pointent vers `…/pb/plateforme/lac/iceberg/…`, le dossier d'origine. L'original effacé, la liste de manifestes elle-même devient introuvable. On ne copie pas une table Iceberg comme un dossier : on la réécrit ailleurs par l'outil qui l'a écrite, ou on la fait lire par son catalogue.

Théorie : `Theorie_10_DE.md`, section 2.

2. Lire les fichiers sans la table, c'est perdre ce que les métadonnées savent. Le renommage n'existe que dans le schéma de la table : les fichiers d'avant mai 2025 portent toujours `temp_moteur_max`, ceux d'après `temperature_moteur_max`, et `union_by_name` en fait deux colonnes. Après le compactage, les 90 fichiers quotidiens de 2026 sont encore sur le disque à côté des 3 fichiers compactés, et la lecture compte deux fois les 46 339 messages de 2026. Seule la table sait quels fichiers comptent.

Théorie : `Theorie_10_DE.md`, sections 4 et 7.

3. Le voyage dans le temps par la date suit l'heure des validations. La table a été écrite en septembre 2026 ; au 31 décembre 2025, elle n'existait pas, et l'état « le plus récent avant cette heure » est une table vide, sans erreur. Il fallait lire l'instantané étiqueté `fin-2025`, ou, sans étiquette, celui dont la période consignée dans le résumé finit au 2025-12-31 : 158 146 messages.

Théorie : `Theorie_10_DE.md`, section 6.

4. Tous les instantanés sauf le dernier sont oubliés, celui de la fin de 2025 compris : l'état au 31 décembre ne se relit plus par la table. Les données, elles, ne sont pas perdues, puisque l'état courant contient tous les messages ; on les retrouve en filtrant sur `jour`, tant que la table n'a reçu que des ajouts. Ce qui est perdu, c'est l'état tel qu'il était, corrections comprises. Il fallait étiqueter d'abord : l'expiration épargne toujours un instantané étiqueté.

Théorie : `Theorie_10_DE.md`, sections 6 et 7.

5. Un chargement écrit ses fichiers de données avant de valider. Tant que la validation n'a pas eu lieu, aucun instantané ne les cite : ils ont l'air d'orphelins. Le nettoyage à zéro heure les efface, et la validation pointe vers des fichiers qui n'existent plus. Les 72 heures par défaut, comme les trois jours de Spark, laissent à toute écriture en cours le temps de finir.

Théorie : `Theorie_10_DE.md`, section 7.

6. Le nettoyage forcé à zéro heure a effacé tous les fichiers que la version courante ne cite plus : le journal sait encore ce qu'était la version 27, mais ses fichiers ont disparu. Deux façons de l'éviter : garder la rétention par défaut, sept jours, allongée par `delta.deletedFileRetentionDuration` si l'audit remonte plus loin ; ou copier la version dans une table à part avant tout nettoyage, par un clone profond sur Databricks. Iceberg offrirait une troisième voie, l'étiquette.

Théorie : `Theorie_10_DE.md`, sections 6 et 7.

7. L'entrepôt rend ses horodatages à l'heure de la session, `America/Toronto`, et Arrow garde le fuseau dans le type. Iceberg ne connaît qu'un horodatage avec fuseau, rangé en UTC : PyIceberg refuse le type avant même de créer la table. `SET TimeZone = 'UTC'` ne change que l'étiquette du fuseau : l'instant, un nombre de microsecondes depuis 1970, reste le même, et la comparaison de l'étape 8 le prouve, 0 écart.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_10_DE.md`, section 5.

8. Le mois d'une partition se compte en UTC, et l'usine vit à l'heure de l'Est. La partition de mars commence le 28 février à 19 h, heure normale, qui est minuit à Greenwich, et s'arrête le 31 mars à 20 h, heure avancée : elle prend les cinq dernières heures de février, et laisse les quatre dernières de mars à la partition d'avril, 220 lignes. Mars compte par ailleurs 743 heures à l'usine, puisque le passage à l'heure avancée, le 8 mars, en retire une : 743 heures de 55 étiquettes font 40 865 lignes. La partition en porte 743 - 4 + 5 = 744 heures, soit 40 920. Pour compter mars, on filtre sur `heure` dans le fuseau de l'usine, et on laisse la table choisir les partitions.

Théorie : `Theorie_10_DE.md`, section 5.

Questions à développer

A. Un choix défendable est Iceberg, pour trois raisons tirées du labo. Il désigne ses colonnes par identifiant : le renommage du micrologiciel 2.0 s'est fait sans réécrire un fichier, avec la bibliothèque du poste, alors que `deltalake` 1.6 a refusé d'activer la correspondance des colonnes et a laissé deux colonnes à moitié vides. Il partitionne par transformation cachée, et le filtre sur octobre 2025 n'a ouvert qu'un fichier sur 28 sans que la requête connaisse la partition. Il étiquette un instantané, et l'état au 31 décembre a survécu à l'entretien. Du côté des plateformes, d'après leurs documentations consultées le 2026-09-23, Snowflake écrit et lit des tables Iceberg dont il est le catalogue, Databricks crée des tables Iceberg gérées, et Fabric présente une table Iceberg en Delta : Iceberg ne ferme aucune porte. Ce qui ferait changer le choix : une entreprise déjà sur Databricks ou sur Fabric, où Delta est le format par défaut, entretenu par la plateforme, et où UniForm ou OneLake le font lire en Iceberg. Le vrai choix durable est le catalogue : c'est lui qui dira où sont les tables et qui les écrit.

Théorie : `Theorie_10_DE.md`, sections 3, 4 et 8.

B. Le voyage dans le temps n'existe que tant que les fichiers existent. À l'étape 7, l'expiration a rendu 122 instantanés illisibles ; à l'étape 9, le nettoyage de Delta a rendu la version 27 illisible en une commande. Une sauvegarde doit survivre à une erreur d'entretien, à un disque perdu, à une table effacée : l'historique d'une table vit dans la table, sur le même stockage, et meurt avec elle. La documentation de Databricks le dit elle-même : pas d'historique de table comme archive, et sept jours sûrs sans réglage. Face au droit à l'effacement, le voyage dans le temps joue contre la personne : ses données retirées restent lisibles dans les instantanés d'avant, puis dans les fichiers orphelins. Un effacement n'est complet qu'après la suppression, l'expiration des instantanés qui les citent et le nettoyage des fichiers, et une étiquette posée sur un vieil état doit être revue quand elle garde des données qu'on doit effacer.

Théorie : `Theorie_10_DE.md`, sections 6 et 7.

C. Nettoyer une donnée demande de connaître sa source. L'argent des séries de l'usine retire les valeurs de départ de l'historien, applique ses codes de qualité, remet l'heure locale d'aplomb et comprend l'archivage de minuit : c'est un savoir de l'ingénieur de données, qui a lu l'historien au Cours 08. Les sources relationnelles, les commandes de l'ERP par exemple, se nettoient par leur sens métier : un statut de commande, un type de document, une devise. L'ingénieur analytique les connaît, et son `STAGING` fait ce travail, une vue par table source, sans jointure. Si Data Engineering tenait aussi un argent des commandes, deux équipes nettoieraient les mêmes données de deux façons : deux définitions de la commande valide, deux chiffres de ventes qui ne s'accordent pas, et personne pour dire lequel fait foi.

Théorie : `Theorie_10_DE.md`, section 10.

© 2026 iAlexMG. Tous droits réservés.

Mises en situation

1. Coopérative de pêcheurs de Rimouski. On passe du dossier de Parquet à une table Iceberg, chargée sortie par sortie, avec une partition cachée par mois de la date de débarquement. Au printemps, on renomme `poids_kg` en `masse_debarquee_kg` et on ajoute la température de l'eau, en une seule évolution du schéma : les fichiers déjà écrits se relisent sous le nom neuf. Pendant les deux mois de bascule, le chargeur lit chaque fichier par celui des deux noms qu'il porte, comme le `COALESCE` du labo, et écrit toujours sous le nom de la table. On demande à l'outil qui écrit de savoir faire un renommage par identifiant ; c'est ce que `deltalake` 1.6 ne savait pas faire au labo, et c'est un critère de choix aussi important que le format.

Théorie : `Theorie_10_DE.md`, sections 4 et 5.

2. Port de Sept-Îles. À chaque rapport trimestriel publié, on étiquette l'instantané qu'il a lu : les étiquettes survivent à l'expiration, et les chiffres se reproduisent pendant cinq ans. Les instantanés non étiquetés expirent après quelques jours, et le nettoyage efface chaque semaine les fichiers orphelins de plus de trois jours. L'effacement demandé par l'armateur se fait en deux temps : la suppression dans la table courante, puis la réécriture de chaque état étiqueté qui contient les personnes visées, en retirant leurs renseignements ; l'ancien instantané expire, et le nettoyage efface ses fichiers. Les chiffres du rapport ne changent pas si les renseignements retirés n'y entraient pas, ce qu'il faut vérifier et consigner. Le cadre légal appartient à la formation Architecture et Gouvernance.

Théorie : `Theorie_10_DE.md`, sections 6 et 7.

3. Aluminerie de Baie-Comeau. Bronze : les exports de l'historien des cuves tels que livrés, et les extraits mensuels de l'ERP, en tables ouvertes. Argent des séries des cuves : les points nettoyés de leurs valeurs de départ et de leurs codes de qualité, rééchantillonnés à l'heure, tenu par l'ingénieur de données, qui connaît l'historien. Les extraits de l'ERP ne passent pas par un argent de l'ingénieur de données : ils vont de la `RAW` au `STAGING` de l'ingénieur analytique. Or : les étoiles de production et les indicateurs des tableaux de bord, tenus par l'ingénieur analytique, qui joint les séries des cuves et les commandes. Chaque couche se reconstruit à partir de la précédente, et personne ne lit le bronze pour faire l'or.

Théorie : `Theorie_10_DE.md`, sections 9 et 10.

4. (avancé) Quincailleries de Shawinigan. Les tables Iceberg locales règlent ce qui fait mal aujourd'hui : un analyste qui lit un dossier sans transaction ni historique. Elles ne demandent aucun service, se lisent par DuckDB, et restent lisibles par Spark, Snowflake ou Databricks si la chaîne grandit ; leur prix est l'entretien, compactage, expiration, nettoyage, à planifier comme n'importe quel travail. Une plateforme clés en main entretient elle-même ses tables, ajoute les droits, le catalogue et les tableaux de bord, mais se paie à l'usage, demande une équipe pour l'administrer, et range la donnée selon ses choix, Delta par défaut chez les deux. Pour un seul analyste et un seul dossier, la recommandation est Iceberg en local, avec un catalogue SQL et un entretien hebdomadaire ; la plateforme devient raisonnable le jour où plusieurs équipes lisent les mêmes tables, ou où il faut des droits par personne.

Théorie : `Theorie_10_DE.md`, sections 7, 8 et 11.

© 2026 iAlexMG. Tous droits réservés.